Qiskit Fall Fest 2026 — Phase 1 Master Guide

Official Unified Challenge Kit & Operational Reference | SRM University-AP

CRITICAL OPERATIONAL UPDATES & CLARIFICATIONS

- **Extended Deadline:** Wednesday, **October 14, 2026, at 23:59 IST** (Midnight IST).
- **Official Submission Portal:** Submit your GitHub repository URL at **https://qffsrmap2026.com/learning/hackathon**.
- **Official Drive Kit:** https://drive.google.com/drive/folders/1caxlXXeqjOkLmXGlimy_Etw0uPHSwyga.
- **Phase 1 Scope:** Strictly **Processor A & Processor B**. Processor C and the custom Processor D design task are exclusively reserved for Phase 2 offline finalists. You do **NOT** need to implement or run C or D for Phase 1.
- **Processor Modeling Challenge (Participant Freedom):** Pre-compiled .json files are intentionally **not provided**. Deducing, specifying, and creatively designing the coupling graphs in Qiskit is Challenge Step 1.
- **Self-Contained Data & Inputs:** No external dataset files required. All tasks use programmatic data generation or inline parameters.

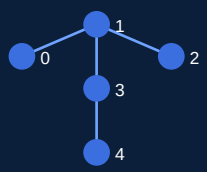
1. Universal Challenge Framework: Algorithm <-> Architecture Co-Design

Quantum algorithms are conceived logically, but real quantum hardware has rigid physical constraints: finite connectivity, limited qubit counts, and noisy two-qubit gates. The objective of Phase 1 is to demonstrate that you can implement an end-to-end quantum solution, map and benchmark it across two distinct physical architectures (**Processor A** and **Processor B**), and systematically evaluate how connectivity, gate routing, circuit depth, and noise impact solution fidelity.

Processor models across stages

Processor A

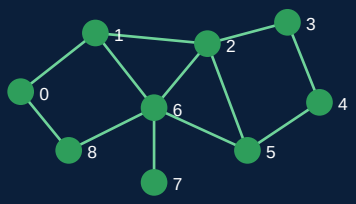
5 qubits - fixed baseline graph



Synthetic noise model

Processor B

9 qubits - heavy-hex-inspired graph



Contemporary compilation baseline

IMPORTANT POLICY: CREATIVE FREEDOM IN COUPLING MAP DESIGN

- The coupling map for a custom processor is **participant-defined and may differ from one team to another**. Participants are free to design and justify any valid connectivity graph within the specified constraints.
- **Why this matters:** There is no single 'correct' coupling map. Different problem structures (e.g. molecular orbitals, graph cut edges, spin chains) benefit from different topologies. Designing your coupling map to minimize SWAP gates for your specific algorithm is where your technical creativity is tested.
- **Constraint:** You must document your chosen coupling graph clearly in your repository (code and visualization) and explain your architectural design decisions.

Processor Specifications

Parameter	Processor A (Baseline)	Processor B (Contemporary)
Physical Qubit Budget	**5 Qubits** (Nodes 0 to 4)	**9 Qubits** (Nodes 0 to 8)
Topology & Coupling Map (Participant-Defined)	**Creative Choice:** You may choose any 5-qubit topology, such as: • Linear chain: [(0,1), (1,2), (2,3), (3,4)] • Star / branching tree: [(0,1), (1,2), (1,3), (3,4)] • Ring / cyclic: 5-cycle. State your choice clearly and explain routing implications.	**Creative Choice:** You may design any sparse 9-qubit graph (planar or heavy-hex-inspired), such as: • Heavy-hex reference (12 edges): (0,1), (1,2), (1,6), (2,3), (2,6), (2,5), (3,4), (4,5), (5,6), (6,7), (6,8), (8,0) • Alternative tree / grid: any valid 9-node graph. Document your chosen graph in `processor_B.json`.
Basis Gate Set	`{rz, sx, x, cx}`	`{rz, sx, x, cx}`
Synthetic Noise Model (for noisy evaluation)	• 1-qubit depolarizing: 0.1% (p=0.001) • 2-qubit CX depolarizing: 0.5% (p=0.005) • Readout error: 1.0% (p=0.01)	Apply identical synthetic noise rates to Processor B so performance differences isolate connectivity effects.
Transpilation Baseline	Optimization Level = 1, Seed = 42, Shots = 4096 (unless PS specifies otherwise)	Optimization Level = 1, Seed = 42, Shots = 4096

2. Participant Workflow & Required Repository Structure

Each team (2 to 6 members) selects **exactly one** problem statement from the 20 available. Submit a single public GitHub repository at **https://qffsrmap2026.com/learning/hackathon** containing:

File / Folder	Required Contents & Description
`main.ipynb` **(or** `main.py`**)**	End-to-end reproducible script: problem formulation, transpilation to A & B, execution, and metric extraction.
`README.md`	Document team members, selected PS code, methodology, quantitative A vs B comparison table, and insights.
`requirements.txt`	Pinned dependency versions (e.g., qiskit>=1.0.0, qiskit-aer, networkx, matplotlib).
`processors/`	Participant-created `processor_A.json` and `processor_B.json` capturing your coupling maps and gate models.
`results/`	Exported figures (PNG) and tables (CSV): circuit depth, 2-qubit gate count, SWAP overhead, and problem objective on A vs B.

3. Detailed Problem Statement Directory (20 Choices Across 5 Tracks)

Below are all 20 challenge choices. Every task is 100% self-contained — no external dataset downloads required.

Track 1: Quantum Chemistry (VQE / Molecular Systems)

PS-C1 Molecular Ground-State Energy (H_2)

TASK	Compute the ground-state energy of H_2 at equilibrium (R = 0.74 Å) using VQE with a hardware-efficient or UCCSD ansatz. Validate against classical diagonalization (approx. -1.137 Hartree). Compare transpilation depth, CX count, and energy accuracy on Processor A vs B.
DATA / SETUP	Use Qiskit Nature with PySCF driver: `Molecule(geometry=[['H', [0., 0., 0.]], ['H', [0., 0., 0.74]]], charge=0, multiplicity=1)`. Alternatively, use the standard 4-qubit Hamiltonian obtained via Jordan-Wigner transformation.

PS-C2 Diatomic Potential Energy Surface (H_2 / LiH)

TASK	Map the potential energy curve across 5-7 bond distances (e.g., 0.5 to 2.5 Å). Keep the variational ansatz fixed and evaluate how transpiled gate counts and optimizer convergence differ between Processor A and B as molecular geometry stretches.
DATA / SETUP	Programmatically iterate bond distances in Python using Qiskit Nature or precomputed electronic integrals.

PS-C3 Hardware-Efficient Triatomic System (H_2O / H_3^+)

TASK	Target a reduced model for a triatomic molecule using active space reduction or qubit tapering to fit within 4-8 logical qubits. Compare ansatz scalability on Processor A vs B.
DATA / SETUP	Generate reduced Hamiltonian via Qiskit Nature active space transformer or freeze-core approximation.

PS-C-OI Open Innovation — Custom Chemistry

TASK	Propose and solve a custom chemistry problem (e.g. excited state calculation, dipole moment, or custom transition metal bond) within 4-8 qubits.
DATA / SETUP	Fully define Hamiltonian generation and exact classical benchmark in your repository.

Track 2: Quantum Optimization (QAOA / QUBO)

PS-O1 Max-Cut on Weighted Graphs

TASK	Solve Max-Cut on a 6-node weighted graph using QAOA (p=1 or p=2). Compare cut value, approximation ratio, circuit depth, and SWAP routing overhead on Processor A vs Processor B.
DATA / SETUP	Generate a 6-node random graph programmatically using NetworkX: `nx.erdos_renyi_graph(n=6, p=0.5, seed=42)` with integer edge weights 1-5.

PS-O2 Portfolio Optimization Under Budget Constraints

TASK	Formulate a 4-asset portfolio selection problem with return maximization and risk penalty: minimize $w^T\Sigma w - q\mu^T w$ subject to $\Sigma w = k$. Encode as QUBO and solve via QAOA or VQE on A vs B.
DATA / SETUP	Use inline expected returns μ = [0.12, 0.18, 0.10, 0.22], budget k = 2, and risk covariance matrix Σ = [[0.04, 0.01, 0.02, 0.01], [0.01, 0.09, 0.01, 0.03], [0.02, 0.01, 0.05, 0.02], [0.01, 0.03, 0.02, 0.08]].

PS-O3 0/1 Knapsack Problem

TASK	Formulate 0/1 Knapsack as a QUBO with quadratic penalty for weight overflow. Solve via QAOA and benchmark against classical brute force. Compare routing cost on A vs B.
DATA / SETUP	Use inline values = [10, 4, 7, 5, 8], weights = [5, 3, 4, 2, 6], and maximum capacity W = 15.

PS-O-OI Open Innovation — Custom Optimization

TASK	Formulate and solve a custom QUBO problem (e.g., TSP, graph coloring, facility location) using QAOA/VQE on A vs B.
DATA / SETUP	Specify problem definition, classical verification, and objective formulation clearly in README.

Track 3: Quantum Simulation (Hamiltonian Dynamics)

PS-S1 Transverse-Field Ising Model (TFIM) Ground State

TASK	Estimate the ground state energy and magnetization of a 4-spin 1D TFIM chain ($H = -J \Sigma Z_i Z_{i+1} - g \Sigma X_i$) across external fields $g \in \{0.5, 1.0, 1.5\}$ using VQE on Processor A vs B.
DATA / SETUP	Construct Hamiltonian directly via Qiskit `SparsePauliOp`: terms 'ZZII', 'IZZI', 'IIZZ' (J=1.0) and 'XIII', 'IXII', 'IIXI', 'IIIX' (coupling g).

PS-S2 Real-Time Trotterized Spin Dynamics

TASK	Simulate real-time unitary evolution $U(t) = \exp(-iHt)$ of the 4-spin TFIM under first-order Trotter decomposition up to t = 2.0. Measure and compare accuracy and SWAP overhead on A vs B.
DATA / SETUP	Initial state |0000> or Néel state |0101>. Compute exact matrix exponential classically via `scipy.linalg.expm` for comparison.

PS-S3 Noise Accumulation in Trotter Circuits

TASK	Study the degradation of Trotterized simulation accuracy as a function of Trotter steps (n=1, 2, 4) under the official synthetic noise model on Processor A vs Processor B.
DATA / SETUP	Same 4-spin TFIM setup. Simulate using Qiskit `AerSimulator` with the specified depolarizing error model.

PS-S-OI Open Innovation — Custom Simulation

TASK	Propose a custom spin chain, fermionic model (Fermi-Hubbard), or time-dependent Hamiltonian simulation on A vs B.
DATA / SETUP	Provide exact numerical diagonalisation baseline in Python.

Track 4: Quantum Machine Learning (QML)

PS-Q1 Variational Quantum Classifier (VQC)

TASK	Build and train a 4-qubit Variational Quantum Classifier with angle encoding and parameterized rotation layers. Compare classification accuracy, training convergence, and compiled circuit depth on Processor A vs B.
DATA / SETUP	Generate 200 synthetic data samples (4 features, binary class) using `sklearn.datasets.make_classification(n_samples=200, n_features=4, n_informative=3, random_state=42)` with an 80/20 train/test split.

PS-Q2 Quantum Kernel Estimator & Support Vector Machine (QSVM)

TASK	Construct a quantum kernel matrix using fidelity state overlap $\|\langle\phi(x)\|\phi(x')\rangle\|^2$ on a 4-qubit register. Train a classical SVM with the quantum kernel. Compare kernel evaluation cost on A vs B.
DATA / SETUP	Use `sklearn.datasets.make_moons(n_samples=100, noise=0.1, random_state=42)`.

PS-Q3 Noise-Resilient Classification & Error Mitigation

TASK	Evaluate a trained VQC under ideal vs synthetic device noise. Apply at least one error mitigation strategy (e.g., Zero-Noise Extrapolation via Mitiq or readout measurement mitigation) on Processor A vs B.
DATA / SETUP	Use the same synthetic classification dataset and official noise parameters.

PS-Q-OI Open Innovation — Custom QML

TASK	Design a custom QML model (e.g. quantum autoencoder, generative QGAN, or hybrid neural net) on A vs B.
DATA / SETUP	Clearly document feature encoding and classical comparison baseline.

Track 5: Post-Quantum Cryptography (Quantum-Safe Systems)

PS-P1 Quantum-Safe Hybrid Key Exchange

TASK	Implement and benchmark a hybrid key encapsulation mechanism combining classical ECDH (P-256) with NIST PQC standard ML-KEM-768 (Kyber). Measure handshake latency, ciphertext size, and memory overhead on simulated standard (Platform A) vs constrained (Platform B) CPU environments.
DATA / SETUP	Use public Python libraries (pyca/cryptography and oqs-python / liboqs or pure-Python ML-KEM reference).

PS-P2 Cryptographic Inventory & Migration Scanner

TASK	Develop a static analysis scanner (AST or pattern-based) that inspects a codebase to detect quantum-vulnerable cryptography (RSA, DSA, DH, ECC) and flags migration paths to NIST FIPS 203/204 standards.
DATA / SETUP	Run the scanner against any public open-source Git repository (e.g. Requests, Flask) and report precision, recall, and scan execution time.

PS-P3 PQC Algorithm Benchmark Across Parameter Sets

TASK	Perform a reproducible empirical benchmark of ML-KEM (512, 768, 1024) and ML-DSA (44, 65, 87) measuring key generation time, encapsulation/signing time, and public key / signature sizes across 100 iterations.
DATA / SETUP	No external dataset required. Use standardized NIST FIPS parameters and generate boxplots/tables.

PS-P-OI Open Innovation — Custom PQC Systems

TASK	Propose a custom quantum-safe systems challenge (e.g. PQC in TLS 1.3, stateful hash signatures XMSS/LMS, or side-channel resistance analysis).
DATA / SETUP	Define threat model and benchmarking setup in repository.

4. Judging Rubric & Phase 1 Participant Checklist

Criteria	Weight	What Judges Look For
Scientific & Algorithmic Correctness	30 pts	Does the algorithm execute without errors? Are results compared against a classical or exact theoretical reference?
Processor A vs B Architectural Analysis & Creativity	35 pts	Quantitative comparison: circuit depth, CX count, SWAP routing overhead, and noise impact. Evaluation of how creatively the participant designed/reasoned about the coupling topologies.
Reproducibility & Code Quality	20 pts	Clean environment, pinned requirements.txt, structured repository, and end-to-end runnable notebook.
Documentation & Visualizations	15 pts	Clear README, coupling map diagrams, and legible performance comparison plots.

FINAL SUBMISSION CHECKLIST (BEFORE OCT 14, 23:59 IST)

- ☐ Exactly **ONE** Problem Statement selected and identified in README.
- ☐ Solution implemented and verified against classical / exact reference.
- ☐ Coupling maps for Processor A (5q) and Processor B (9q) creatively defined, justified, and transpiled.
- ☐ Quantitative comparison table included (Depth, CX Count, SWAP overhead, Accuracy on A vs B).
- ☐ Public GitHub repository submitted at https://qffsrmap2026.com/learning/hackathon.

Note: Processor C and Processor D are NOT part of Phase 1 and are not required.